

Development environment and package management

➔ Managing python packages with pip

As our code project increases we may have to include more and more python packages but managing these packages is kind of challenging

Pip basics:

Using pypi packages repository

Pip command always is used inside active env

This command installs the **Arrow** Python library, a tool for managing dates and times.

It utilizes the pip package manager specifically aimed at the Python interpreter.

Ensure you have a terminal open and Python installed to run this successfully.

```
python -m pip install arrow
```

Once the installation is complete you can use show command that shows the meta

data about the library

```
$ python -m pip show arrow
```

 importantly version and location is it installed in .

```
python -m pip list
```

 command to view all the packages installed.

Installing specific versions by using pip :

Requirement specifier : requirement specifier is composed of a project name followed by optional version specifiers.

Surrounding the requirement specifier helps us prevent the possibility of having an error

Packages with dependencies :

When installing any module all the dependencies associated or mentioned in the source code of that package is going to be installed

When installing fastapi we are installing many dependencies

```
Installing collected packages: typing-extensions, sniffio, idna, pydantic, anyio, starlette, fastapi
```

Dependencies can have their own dependencies

There exist some optional dependencies unlike normal dependencies , these are not installed directly , we have to specifical install them

To install these extras you have use square brackets and you have to specify the

name

```
python -m install fastapi[doc,dev]
```

You can view everything about requirement specifiers , by using a document PEP508

This reports shows everything about packages dependencies and extras as well

In PEP 440 we have all the information about version identification and dependency specification

Version specifiers

A version specifier consists of a series of version clauses, separated by commas. For example:

```
~ 0.9, >= 1.0, != 1.3.4.*, < 2.0
```

The comparison operator determines the kind of version clause:

- `~`: Compatible release clause
- `==`: Version matching clause
- `!=`: Version exclusion clause
- `<=`, `>=`: Inclusive ordered comparison clause
- `<`, `>`: Exclusive ordered comparison clause
- `===`: Arbitrary equality clause.

The comma (",") is equivalent to a logical **and** operator: a candidate version must match all given version clauses in order to match the specifier as a whole.

Whitespace between a conditional operator and the following version identifier is optional, as is the whitespace around the commas.

When multiple candidate versions match a version specifier, the preferred version **SHOULD** be the latest version as determined by the consistent ordering defined by the standard [Version scheme](#). Whether or not pre-releases are considered as candidate versions **SHOULD** be handled as described in [Handling of pre-releases](#).

Except where specifically noted below, local version identifiers **MUST NOT** be permitted in version specifiers, and local version labels **MUST** be ignored entirely when checking if candidate versions match a given version specifier.

Compatible release

Sometimes we have installed something that is not in official python package repository .that means package from github

Steps to install the package from github directly :

1. Copy the http link of the repository
2. This command uses pip to install a Python package directly from a GitHub repository.The package is named demo_pkg and is sourced from the codesensei-courses GitHub account.It specifically installs the code from the demo_pkg.git repository

```
python -m pip install "demo_pkg @ git+https://github.com/codesensei-courses/demo_pkg.git"
```

3. You can check and can take help from pip documentation under vcs support .

Installing Packages – Requirements Specifiers 2

```
# Compatible release:
# Same major version, minor version equal or greater
pip install "requests ~=2.24.0"

# Equivalent
pip install "requests >=2.24, ==2.*"

# Environment marker
pip install "asyncio; python_version=='3.4'"
pip install "typing; python_version<'3.5'"

pip install "asyncio; os_name=='posix'"      # Mac OS, Linux, UNIX
pip install "asyncio; os_name='nt'"          # Windows
pip install "asyncio; sys_platform='darwin'" # Mac OS
```

Installing from Other Sources

```
# Install from Github URL
pip install "MyProject @ git+ssh://git.example.com/MyProject"
pip install "MyProject @ git+https://git.example.com/MyProject"

# Install from commit, tag or branch
pip install "MyProject @ git+https://git.example.com/MyProject@master"
pip install "MyProject @ git+https://git.example.com/MyProject@da39b"
pip install "MyProject @ git+https://git.example.com/MyProject@v2.1"

# Install a local project or wheel file
pip install path/to/my/project
pip install myproject-1.1.whl
```

- ❑ Using the `--user` flag with `pip install` allows you to install Python packages specifically for your current user account rather than globally for the entire system.
- ❑ This approach is often used for installing utility tools or packages that are not tied to a specific project and do not require administrative privileges.
- ❑ Packages installed this way are typically stored in a user-specific `site-packages` directory

For inbuilt python env operating system that is linux it doesn't exist in windows or mac , we have to install python in windows and mac

Configuring pip: it is difficult to do all the pips so it is easier to create config file

- Hosting your own package repo
- Using a proxy server
- Custom SSL certificates

We can create config file in three places:

1. global: system-wide configuration file, shared across users.
2. user: per-user configuration file.
3. site: per-environment configuration file; i.e. per-virtualenv.

How to works with venv ad vscode

To work with python file you must have python installed

On vscode to run python code , you have setup venv yourself and select python interpreter

Managing dependencies for application :

- ➔ deployed by yourself
- ➔ requirement files

for getting information about requirement file you can get from pip documentation

in requirements file we can also add the github repo package for installation

it is common to have more than one requirements file

so that you can define all the file for production and testing differently

managing dependencies for library not yourself :

- ➔ not deployed yourself
- ➔ distributed to others
- ➔ Pyproject.toml file - The pyproject.toml file is a modern standard in Python for defining project metadata, build settings, and dependencies.
- ➔ Don't pin versions

If you are writing a package to distribute and used by others , be it inside the company or to the entire world

If the version of a module is not matching with the version of another module or library it uses the technique of backtracking .

Handling dependency conflicts :

You can handle the dependencies conflicts either losing the version used or changing the dependencies